# OpenClaw：搭建你的量化交易同事

# >> 今天的学习目标

## OpenClaw：搭建你的量化交易同事

- OpenClaw安装配置

OpenClaw安装前置要求

源码安装步骤

初始化配置（Onboarding）

飞书集成配置

构建 Control UI（网页界面）

配置阿里云模型（OpenAI 兼容模式）

运行与使用

- OpenClaw技能搭建

量化技能实现

miniqmt-kline，talib，backtrader

自定义技能看不到怎么办?

OpenClaw技能使用

如何新增一个 Skill

# OpenClaw安装

# Step1： OpenClaw安装前置要求

- OpenClaw安装前置要求：

| 组件 | 要求 | 验证命令 |
| --- | --- | --- |
| Node.js | 需安装 npm 和 node | npm -v / node -v |
| Git for Windows | 必须安装，用于 bash 脚本 | where.exe git |
| PowerShell | Windows 自带 | - |
| Git Bash | npm 构建需要 bash 环境 | 需配置路径 |

# Node.js安装

Thinking：什么是Node.js?

JavaScript 运行时环境，包含 npm（Node Package Manager）。OpenClaw 是基于 Node.js 开发的，需要它来运行代码和管理依赖。

Thinking：Node.js在 OpenClaw 中的作用是什么?

运行 npm install 安装项目依赖（约 1000+ 个包）

执行 npm run build 构建项目

通过 npm link 将 openclaw 注册为全局命令

Thinking：如何下载安装Node.js?

官网：https://nodejs.org/

建议下载 LTS（长期支持版），如 v20.x 或 v22.x，安装时勾选 Add to PATH（自动添加到环境变量）

验证node是否成功：

node -v # 显示版本如 v22.12.0

npm -v # 显示版本如 11.3.0

# Git for Windows安装

Thinking：什么是Git?

分布式版本控制系统。最重要的是它包含了 Git Bash（Unix 风格的命令行工具）。

Thinking：Git在 OpenClaw 中的作用是什么?

OpenClaw 的构建脚本（scripts/bundle-a2ui.sh 等）是 Bash 脚本，必须在 Bash 环境中运行

通过 git clone 下载源码

提供 bash.exe 供 npm 调用（关键：npm config set script-shell）

Thinking：如何下载安装Node.js?

官网：https://git-scm.com/download/win

国内镜像（较快）：https://registry.npmmirror.com/binary.html?path=git-for-windows/

验证git是否成功：

```
where.exe git
# 应输出如：D:\Program Files\Git\cmd\git.exe
```

# Git Bash使用

Thinking：什么是Git Bash?

Git for Windows 附带的 Unix/Linux 风格的命令行终端，提供了 Bash shell 和常用的 Linux 命令（如 ls, cat, grep）。

Thinking：Git Bash在 OpenClaw 中的作用是什么?

执行项目中的 .sh 构建脚本

npm 需要通过它来运行 bash scripts/bundle-a2ui.sh

Thinking：如何配置Git Bash路径?

安装 Git for Windows 后，需要手动配置 npm 使用 Git Bash：

```
# 默认路径通常是：
npm config set script-shell "C:\Program Files\Git\bin\bash.exe"
# 或你的自定义路径：
npm config set script-shell "D:\Program Files\Git\bin\bash.exe"
```

验证Git Bash是否成功：

```
Test-Path "C:\Program Files\Git\bin\bash.exe"
# 返回 True 表示路径正确
```

# PowerShell使用

Thinking：什么是PowerShell?

Windows 自带的现代化命令行 Shell，比传统的 CMD 更强大，支持脚本编程。

Thinking： PowerShell在 OpenClaw 中的作用是什么?

执行所有安装命令

设置环境变量（如 $env:OPENAI_API_KEY="...")

运行 openclaw 系列命令

Windows 10/11 已默认安装 PowerShell

# 快速检查清单（前置安装软件）

```
# 1. 检查 Node.js
node -v

# 2. 检查 npm
npm -v

# 3. 检查 Git
git --version

# 4. 检查 Git Bash 路径（根据你的安装位置）
Test-Path "C:\Program Files\Git\bin\bash.exe"
# 如果不存在，尝试查找：
where.exe bash
```

# Step2：源码安装步骤

- 克隆源码并进入目录

```
git clone https://github.com/openclaw/openclaw.git
cd openclaw
```

- 配置 npm 使用 Git Bash（关键步骤）

OpenClaw 的构建脚本需要 bash 环境，需将 npm 的 script-shell 指向 Git Bash：

```
# 验证 Git Bash 路径（根据你的实际安装路径调整）
Test-Path "D:\Program Files\Git\bin\bash.exe"

# 配置 npm 使用 bash
npm config set script-shell "D:\Program Files\Git\bin\bash.exe"
```

# Step2：源码安装步骤

- 安装依赖并构建

```
# 安装依赖（约 1000+ 包，可能需要几分钟）
npm install
# 构建项目（会生成 dist 目录）
npm run build
# 全局链接，使命令行可用
npm link
```

- 验证安装

```
openclaw --version
# 应显示：2026.3.3
```

管理员: Windows PowerShell

```
✔Build complete in 1846ms
✔Build complete in 1848ms
ℹdist\bundled\session-memory\handler.js          10.69 kB │ gzip:   3.70 kB
ℹdist\bundled\boot-md\handler.js                  8.28 kB │ gzip:   2.95 kB
ℹdist\llm-slug-generator.js                       4.87 kB │ gzip:   2.10 kB
ℹdist\bundled\bootstrap-extra-files\handler.js    2.18 kB │ gzip:   0.90 kB
ℹdist\bundled\command-logger\handler.js           1.66 kB │ gzip:   0.80 kB
ℹdist\pi-embedded-BHF5v-o7.js                     3.43 MB
ℹdist\model-selection-ikt2OC4j.js               572.45 kB │ gzip: 123.48 kB
ℹdist\pi-embedded-helpers-DvNaskDY.js           255.03 kB │ gzip:  58.85 kB
ℹdist\manager--qQfR6iR.js                       141.22 kB │ gzip:  31.70 kB
ℹdist\send-Mhq_NgNz.js                          109.69 kB │ gzip:  26.25 kB
ℹdist\send-BjxxplhK.js                           93.56 kB │ gzip:  24.29 kB
ℹdist\web-DMbkFJSQ.js                            85.06 kB │ gzip:  21.77 kB
ℹdist\sessions-_Cu6BqJv.js                       83.23 kB │ gzip:  20.77 kB
ℹdist\chrome-BJ1olan3.js                         81.31 kB │ gzip:  20.71 kB
ℹdist\audio-transcription-runner-DYom1Phd.js     73.21 kB │ gzip:  17.25 kB
ℹdist\image-5e0tgQ1Z.js                          66.23 kB │ gzip:  16.79 kB
ℹdist\pw-ai-CPlpJFd0.js                          66.05 kB │ gzip:  16.24 kB
ℹdist\deliver-FEreRY0J.js                        57.75 kB │ gzip:  14.17 kB
ℹdist\workspace-BU8QxCQK.js                      53.04 kB │ gzip:  11.71 kB
ℹdist\qmd-manager-Bal1OEC6.js                    52.86 kB │ gzip:  13.42 kB
ℹdist\skills-D3c_mBu8.js                         41.62 kB │ gzip:  11.03 kB
ℹdist\commands-registry-D7j4kc5h.js              34.38 kB │ gzip:   7.77 kB
ℹdist\dock-D49biwXh.js                           34.27 kB │ gzip:   6.85 kB
ℹdist\subsystem-nlluZawe.js                      30.34 kB │ gzip:   8.44 kB
ℹdist\ir-B6xHv9oi.js                             28.21 kB │ gzip:   7.20 kB
ℹdist\query-expansion-CeyLNalm.js                21.26 kB │ gzip:   7.34 kB
ℹdist\registry-ycXZ0GNV.js                       20.54 kB │ gzip:   5.73 kB
```

# Step3：初始化配置（Onboarding）

- 启动配置向导

```
openclaw onboard --install-daemon
```

- 配置选项选择

按交互提示选择：

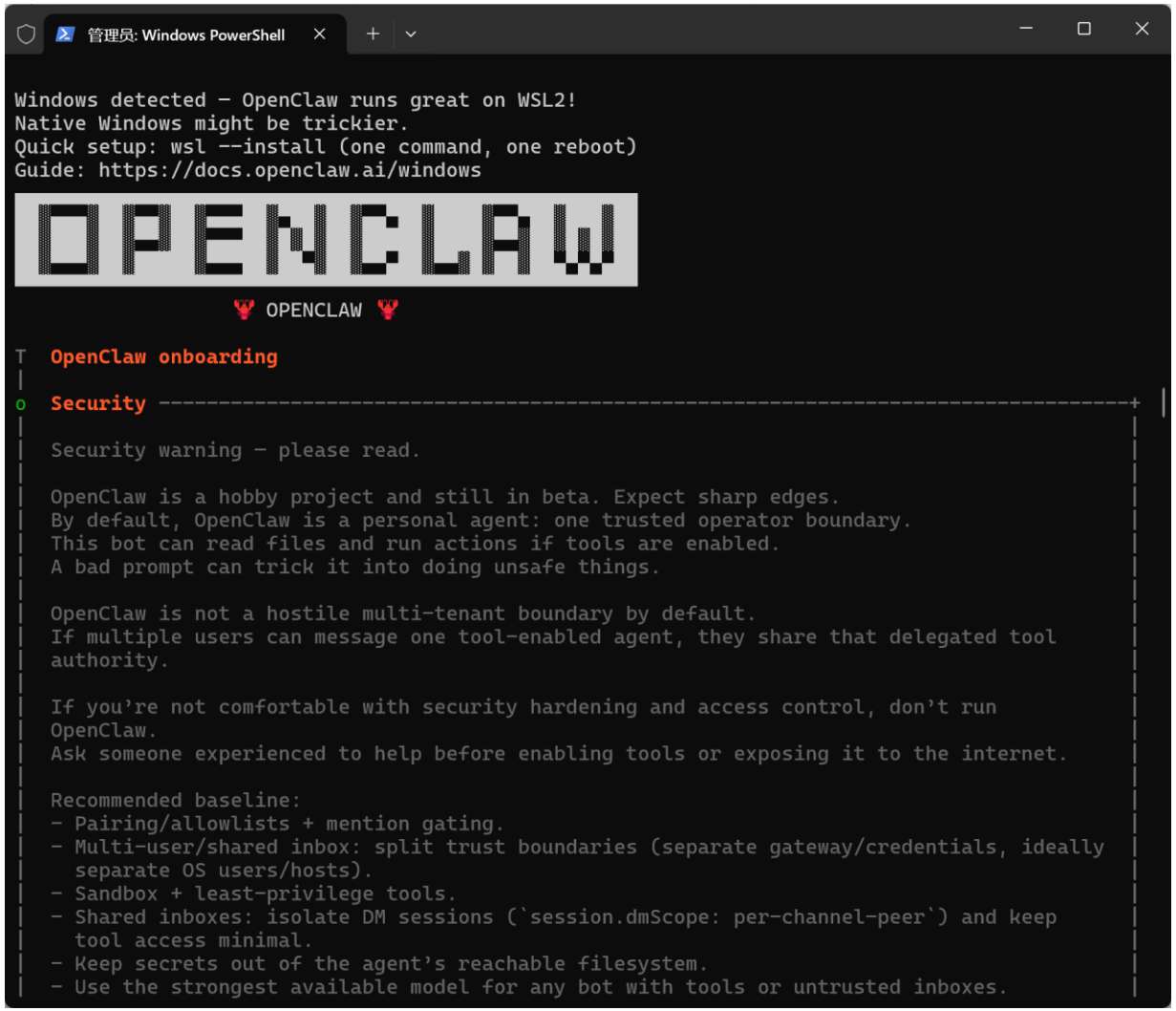
Security: Yes（确认理解安全警告）

Onboarding mode: Manual（手动配置）

Setup: Local gateway (this machine)（本地网关）

Workspace: D:\.openclaw\workspace（或自定义路径）

Model/auth provider: Qwen（通义千问）



# Step3：初始化配置（Onboarding）

- 网关基础配置

Port: 18789（默认）

Bind: Loopback (127.0.0.1)（仅本地访问）

Auth: Token（自动生成令牌）

# Step4：飞书集成配置

- 安装飞书插件

如果在向导中遇到 Error: spawn npm ENOENT，手动安装：

```
npm install -g @openclaw/feishu
```

- 配置飞书凭证

运行配置向导：

```
openclaw configure
```

选择：

- Channels → Feishu/Lark → Use local plugin path（或自动检测）
- App ID: 你的飞书应用 ID
- App Secret: 你的飞书应用密钥
- Domain: Feishu (feishu.cn)（国内版）
- Group policy: Open（需在群组中 @机器人才能响应）

# Step4：飞书集成配置

- 飞书后台配置要求

确保飞书开放平台已配置：

- 开启机器人能力
- 订阅方式设置为：长连接接收事件
- 权限：im:message、im:chat、contact:user.base:readonly

# Step5：构建 Control UI（网页界面）

- 如果在运行时发现 Control UI assets missing，需手动构建：

```
# 进入 UI 目录
cd ui
# 安装 UI 依赖
npm install
# 构建生产版本
npm run build
# 返回上级目录
cd ..
```

构建完成后，会在 dist/control-ui/ 生成网页文件。

# Step6：配置阿里云模型（OpenAI 兼容模式）

- 编辑配置文件

C:\Users\{用户名}\.openclaw\openclaw.json，配置 DashScope（阿里云）的 OpenAI 兼容 API：

```
{
 "models": {
  "providers": {
   "dashscope": {
    "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
    "apiKey": "sk-xxx",
    "api": "openai-completions",
    "models": [
     { "id": "qwen-max-latest", "name": "Qwen Max Latest" }
    ]
   }
  }
 },
```

```
 "agents": {
  "list": [
   {
    "id": "quant-analyst",
    "model": "dashscope/qwen-max-latest",
    "workspace": "~/.openclaw/workspace",
    "skills": ["miniqmt-kline", "talib", "backtrader"]
   }
  ]
 }
}
```

# Step6：配置阿里云模型（OpenAI 兼容模式）

| 配置项 | 说明 |
| --- | --- |
| models.providers.dashscope | 接入阿里云百炼的 Qwen 模型，走 OpenAI 兼容接口 |
| agents.list[0].id | Agent 的唯一标识，命令行用 --agent quant-analyst 指定 |
| agents.list[0].model | Agent 使用的 LLM 模型，格式为 provider/model-id |
| agents.list[0].skills | Agent 加载的技能列表，对应 skills/ 下的文件夹名 |

# Step6：配置阿里云模型（OpenAI 兼容模式）

- 环境变量方式（可选）

也可以在启动网关前设置环境变量：

```
$env:OPENAI_API_KEY="sk-你的API_KEY"
$env:OPENAI_BASE_URL="https://dashscope.aliyuncs.com/compatible-mode/v1"
$env:OPENCLAW_MODEL="openai:qwen-turbo"
```

# Step7：运行与使用

- 启动网关服务

```
openclaw gateway --port 18789
```

看到 listening on ws://127.0.0.1:18789 即表示成功。

- 打开控制面板

新开启一个 PowerShell 窗口：

```
openclaw dashboard
```

会自动打开浏览器：http://127.0.0.1:18789/#token=...

```
🦞 OpenClaw 2026.3.3 (75775f2) — Your config is valid, your assumptions are not.

01:26:34 [gateway] [plugins] plugins.allow is empty; discovered non-bundled plugins may auto-load: feishu
事\openclaw\extensions\feishu\index.ts). Set plugins.allow to explicit trusted ids.
01:26:57 [gateway] feishu_doc: Registered feishu_doc, feishu_app_scopes
01:26:57 [gateway] feishu_chat: Registered feishu_chat tool
01:26:57 [gateway] feishu_wiki: Registered feishu_wiki tool
01:26:57 [gateway] feishu_drive: Registered feishu_drive tool
01:26:57 [gateway] feishu_bitable: Registered bitable tools
01:26:57 [canvas] host mounted at http://127.0.0.1:18789/__openclaw__/canvas/ (root C:\Users\cheny\.opencl
01:26:58 [heartbeat] started
01:26:58 [health-monitor] started (interval: 300s, startup-grace: 60s, channel-connect-grace: 120s)
01:26:58 [gateway] agent model: openai/qwen-turbo
01:26:58 [gateway] listening on ws://127.0.0.1:18789, ws://[::1]:18789 (PID 69180)
01:26:58 [gateway] log file: \tmp\openclaw\openclaw-2026-03-07.log
09:26:58 [info]: [ 'client ready' ]
01:26:58 [browser/server] Browser control listening on http://127.0.0.1:18791/ (auth=token)
01:26:58 [feishu] starting feishu[default] (mode: websocket)
01:26:58 [feishu] feishu[default]: bot open_id resolved: ou_4a8f06f36c8d0926622af63020de311b
09:26:58 [info]: [ 'event-dispatch is ready' ]
01:26:58 [feishu] feishu[default]: starting WebSocket connection...
09:26:58 [info]: [
  '[ws]',
  'receive events or callbacks through persistent connection only available in self-build & Feishu app, Co
    '        Developer Console(开发者后台) \n' +
    '          ->\n' +
    '        Events and Callbacks(事件与回调)\n' +
    '          -> \n' +
    '        Mode of event/callback subscription(订阅方式)\n' +
    '          -> \n' +
    '        Receive events/callbacks through persistent connection(使用 长连接 接收事件/回调)'
]
01:26:58 [feishu] feishu[default]: WebSocket client started
09:26:58 [info]: [ '[ws]', 'ws client ready' ]
```

# Step7：运行与使用

- 使用方式

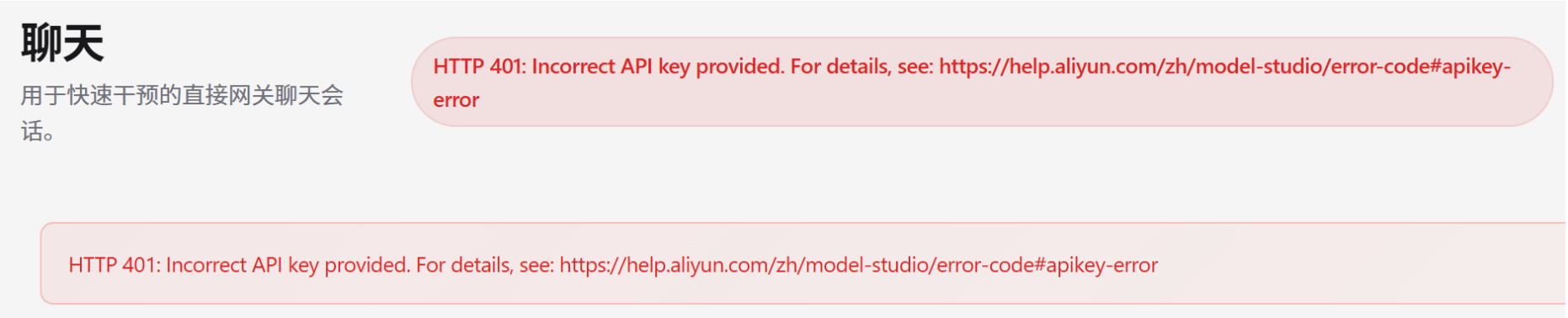
网页聊天：在打开的 Control UI 中直接对话

飞书聊天：在飞书群组中 @机器人发送消息

API 调用：通过 WebSocket ws://127.0.0.1:18789 连接

# 关于环境变量

Thinking：如果页面上报了API KEY Error，怎么办？




检查环境变量的设置

$env:OPENAI_BASE_URL="https://dashscope.aliyuncs.com/compatible-mode/v1"

$env:OPENCLAW_PROVIDER="openai"

$env:OPENAI_API_KEY="sk-your_api_key"

```
$env:OPENAI_BASE_URL="https://dashscope.aliyuncs.com/compatible-mode/v1

$env:OPENCLAW_PROVIDER="openai"
$env:OPENAI_API_KEY="sk-[redacted]
openclaw gateway --port 18789
```

# 关于环境变量

OpenClaw 中可以配置.env（优先级从高到低）：

1. 当前工作目录 (CWD) 下的 .env（dotenv 默认从 process.cwd() 查找）

2. 全局 ~/.openclaw/.env（即 $OPENCLAW_STATE_DIR/.env）

如果没有在项目根目录执行命令，或路径含中文等原因导致 CWD 下的 .env 未被加载

=> API 密钥无法生效。

方案 A：设置 Windows 用户级持久环境变量（推荐）

方案 B：创建全局 .env

在 C:\Users\<用户名>\.openclaw\.env 中写入上述变量，作为全局备选。

# openclaw.json配置

Thinking：启动 gateway 时报错：

```
Invalid config at openclaw.json:
- models: Unrecognized keys: "default", "flash", "max"
- agents: Unrecognized key: "quant-analyst"
```

OpenClaw 2026.3.3 的配置格式已更新，不再使用 models.default、models.flash 等扁平结构，也不支持 agents.quant-analyst 这种对象形式。

正确格式：

models：改为 models.providers，每个 provider 需包含 baseUrl、apiKey、api、models 数组。

agents：改为 agents.list 数组，每个 agent 需包含 id、model、skills 等。

# openclaw.json配置

```
{
  "gateway": {
    "mode": "local",
    "port": 18789,
    "auth": { "token": "${OPENCLAW_GATEWAY_TOKEN}" }
  },
  "models": {
    "mode": "merge",
    "providers": {
      "dashscope": {
        "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
        "apiKey": "${OPENAI_API_KEY}",
        "api": "openai-completions",
        "models": [
          { "id": "qwen-turbo", "name": "Qwen Turbo" },
          { "id": "qwen-turbo-latest", "name": "Qwen Turbo Latest" },
          { "id": "qwen-max-latest", "name": "Qwen Max Latest" }
        ]
      }
    }
  },
  "agents": {
    "defaults": {
      "model": { "primary": "dashscope/qwen-turbo" },
      "workspace": "~/.openclaw/workspace"
    },
    "list": [
      {
        "id": "quant-analyst",
        "model": "dashscope/qwen-turbo-latest",
        "workspace": "~/.openclaw/workspace",
        "skills": ["miniqmt-kline", "talib", "backtrader"]
      }
    ]
  }
}
```

openclaw.json

# Summay（常见问题解决）

| 问题 | 解决 |
| --- | --- |
| openclaw 命令未识别 | 确保运行了 npm link |
| spawn npm ENOENT | 手动全局安装插件：npm install -g @openclaw/xxx |
| Control UI assets missing | 进入 ui 目录运行 npm install + npm run build |
| agents.defaults.model: Invalid input | 检查配置文件中的模型 ID 格式是否为 provider/model |
| JSON5 parse failed | 配置文件 JSON 格式错误，检查逗号和括号匹配 |
| HTTP 403 forbidden | API Key 无效或模型未授权，检查阿里云 DashScope 控制台 |

# Summay

- 配置文件目录结构

C:\Users\{用户名}\.openclaw\

```
├── openclaw.json        # 主配置文件
├── openclaw.json.bak    # 自动备份
├── workspace\           # 工作目录
├── canvas\              # 画布文件
```

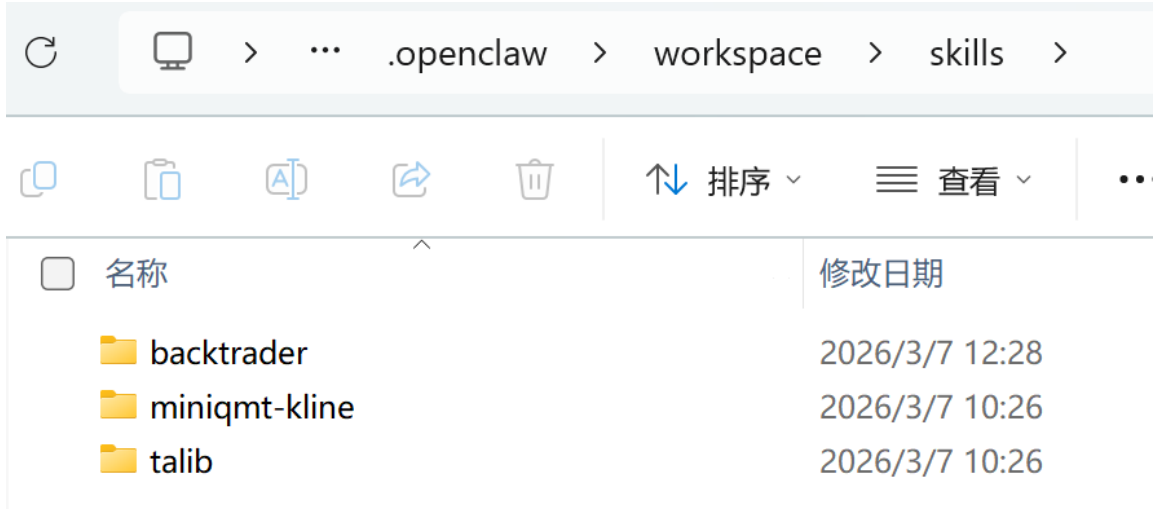
你可以通过网页 http://127.0.0.1:18789 或飞书与 OpenClaw 进行交互。

# OpenClaw技能配置与使用

# 量化技能实现

基于已有代码，我们可以创建三个 OpenClaw Skill

```
skills/
├── miniqmt-kline/
│   ├── SKILL.md
│   └── scripts/
│       └── get_kline.py
├── talib/
│   ├── SKILL.md
│   └── scripts/
│       └── calc_indicators.py
└── backtrader/
    ├── SKILL.md
    └── scripts/
        └── run_backtest.py
```




每个技能都是独立的文件夹，包含一个 SKILL.md 和一个 scripts/ 子目录。这种结构使得技能可以独立开发、测试和部署。

# 量化技能：miniqmt-kline

miniqmt-kline 技能：

- 功能：通过 MiniQMT(xtquant) 获取 A 股 K 线数据
- 前置：MiniQMT 已登录，xtquant 已配置
- 用法：python3 scripts/get_kline.py 600519.SH 1d 100

Thinking：Skill的执行逻辑是怎样的？

获取茅台K线 —> Gateway —> Agent(LLM) —> 匹配 Skill —> exec 工具 —> get_kline.py —> JSON

# 量化技能：miniqmt-kline

Thinking：如何定义SKILL.md?

## 第一部分：YAML 元数据

```
---
name: miniqmt-kline
description: "获取 A 股 K 线。禁止 pip install、禁止检查环境。唯一正确做法：
 直接执行 exec，命令 python skills/miniqmt-kline/scripts/get_kline.py ..."
metadata:
 openclaw:
  emoji: "📈"
  os: ["win32"]                # 支持的操作系统
  requires:
   bins: ["python"]            # 需要的可执行程序
   packages: ["xtquant", "pandas"]  # 需要的 Python 包（仅声明，不自动安装）
  permissions:
   - "exec"                    # 需要执行命令的权限
  timeout: 60                  # 超时时间（秒）
```

# 量化技能：miniqmt-kline

第二部分：Markdown 正文

```
## 执行流程（必须遵守）

1. 收到 K 线请求后，直接执行 exec，无需检查 pip、xtquant、python 版本等
2. 命令：python skills/miniqmt-kline/scripts/get_kline.py <stock_code> [period] [count]
3. workdir 为工作区根目录
4. 将脚本输出的 JSON 直接作为结果返回给用户

禁止：安装依赖、检查环境、验证包、调用 xtquant API。
```

正文是给 AI 看的操作手册，包含执行流程、参数说明、触发示例等。关键是执行流程部分

# 量化技能：miniqmt-kline

## get_kline.py -- K 线数据获取

```
from xtquant import xtdata
def get_kline_data(stock_code, period='1d', count=100):
    # 1. 连接 MiniQMT 客户端
    xtdata.connect()

    # 2. 下载历史数据（增量更新，不会重复下载）
    xtdata.download_history_data(stock_code, period=period, incrementally=True)

    # 3. 获取市场数据（OHLCV + 成交额）
    data = xtdata.get_market_data_ex(
        field_list=['open', 'high', 'low', 'close', 'volume', 'amount'],
        stock_list=[stock_code],
        period=period,
        count=count
    )
```

```
    # 4. 转换为标准 JSON 格式
    df = data[stock_code]
    records = [{"date": ..., "open": ..., "close": ..., ...} for r in df]
    return {"stock_code": stock_code, "period": period, "data":
records}
```

- xtdata.connect() 需要 MiniQMT 客户端在后台运行
- download_history_data 做增量更新，首次运行会较慢
- get_market_data_ex 是 xtquant 的核心 API，返回 DataFrame
- 输出统一为 JSON，方便 AI 解析和展示

工作流：连接 MiniQMT --> 下载历史数据 --> 获取市场数据 --> 格式化为 JSON

# 量化技能：talib

talib 技能：

- 功能：计算 SMA、EMA、MACD、RSI、ATR、BBANDS 等技术指标
- 用法：

 - python3 scripts/calc_indicators.py macd --stock 600519.SH

 - python3 scripts/calc_indicators.py rsi data.csv --period 14

Thinking：Skill的执行逻辑是怎样的？

计算MACD —> Gateway —> Agent(LLM) —> 匹配 Skill —> exec 工具 —> calc_indicators.py —> JSON

# 量化技能：talib

calc_indicators.py -- 技术指标计算

```
import talib
# 支持的指标注册表
INDICATORS = {
    "sma":  (calc_sma,  ["close"],          {"period": 10}),
    "ema":  (calc_ema,  ["close"],          {"period": 12}),
    "macd": (calc_macd, ["close"],          {"fast": 12, "slow": 26, "signal": 9}),
    "rsi":  (calc_rsi,  ["close"],          {"period": 14}),
    "atr":  (calc_atr,  ["high", "low", "close"], {"period": 14}),
    "bbands":(calc_bbands,["close"],        {"period": 20}),
}
# 以 MACD 为例
def calc_macd(close, fast=12, slow=26, signal=9):
    macd, sig, hist = talib.MACD(close, fastperiod=fast, slowperiod=slow,
signalperiod=signal)
    return {"macd": macd.tolist(), "signal": sig.tolist(), "histogram": hist.tolist()}
```

- 使用注册表模式（INDICATORS 字典），新增指标只需加一行
- 两种数据来源：--stock 600519.SH（从 MiniQMT 获取）或直接传 CSV 路径
- TA-Lib 的函数都接收 numpy 数组，返回 numpy 数组
- 输出截取最后 30 条，避免数据量过大；同时输出 last_values 展示最新值

工作流：加载数据（CSV 或 MiniQMT）--> 调用 TA-Lib 计算指标 --> 输出 JSON

# 量化技能：backtrader

Backtrader技能：

- 功能：从 MySQL trade_stock_daily 加载数据，运行双均线/MACD/RSI/布林带等策略回测
- 前置：MySQL 已配置，表 trade_stock_daily 有数据
- 用法：python3 scripts/run_backtest.py 600519.SH 2024-01-01 2025-12-31 double_ma
- 策略：double_ma、macd、rsi、bbands、momentum
- 配置：需设置 BACKTEST_CONFIG_DIR 指向含 .env 的目录，或在该目录下运行

Thinking：Skill的执行逻辑是怎样的？

双均线回测 —> Gateway —> Agent(LLM) —> 匹配 Skill —> exec 工具 —> run_backtest.py —> JSON

# 量化技能：talib

## run_backtest.py -- 策略回测

```
import backtrader as bt
# 双均线策略示例
class DoubleMAStrategy(bt.Strategy):
    params = (("fast", 10), ("slow", 30))

    def __init__(self):
        self.ma_fast = bt.indicators.SMA(self.data.close, period=self.p.fast)
        self.ma_slow = bt.indicators.SMA(self.data.close, period=self.p.slow)
        self.crossover = bt.indicators.CrossOver(self.ma_fast, self.ma_slow)

    def next(self):
        if not self.position:
            if self.crossover > 0:  # 金叉：短均线上穿长均线，买入
                self.buy()
        elif self.crossover < 0:    # 死叉：短均线下穿长均线，卖出
            self.close()
```

```
def run_backtest(stock_code, start_date, end_date, strategy_name):
    df = load_stock_data(stock_code, start_date, end_date)  # 从 MySQL 加载
    cerebro = bt.Cerebro()
    cerebro.addstrategy(strat_cls)          # 添加策略
    cerebro.adddata(bt.feeds.PandasData(dataname=df))  # 添加数据
    cerebro.broker.setcash(1000000)         # 初始资金 100 万
    cerebro.broker.setcommission(commission=0.0002)   # 手续费万 2
    cerebro.addsizer(bt.sizers.PercentSizer, percents=95)  # 95% 仓位

    # 添加分析器
    cerebro.addanalyzer(bt.analyzers.SharpeRatio, _name="sharpe")
    cerebro.addanalyzer(bt.analyzers.DrawDown, _name="drawdown")
    cerebro.addanalyzer(bt.analyzers.TradeAnalyzer, _name="trades")

    results = cerebro.run()
    # 提取绩效指标...
    return {"total_return": ..., "max_drawdown": ..., "sharpe_ratio": ..., ...}
```

# 量化技能：talib

run_backtest.py -- 策略回测

- 5 种内置策略：double_ma、macd、rsi、bbands、momentum
- 数据来源是 MySQL trade_stock_daily 表，通过 pymysql 连接
- .env 加载优先级：BACKTEST_CONFIG_DIR 指定目录 > 脚本所在技能目录 > 当前工作目录
- 输出包含：总收益率、年化收益率、最大回撤、夏普比率、交易次数、胜率

加载 .env 配置 --> 从 MySQL 读取 K 线 --> 构建 Backtrader 引擎 --> 运行策略 --> 输出绩效

# Summary

| 维度 | miniqmt-kline | talib | backtrader |
| --- | --- | --- | --- |
| 用途 | 获取 A 股 K 线数据 | 计算技术指标 | 运行策略回测 |
| 数据来源 | MiniQMT 客户端（xtquant） | MiniQMT 或 CSV 文件 | MySQL 数据库 |
| 核心依赖 | xtquant, pandas | talib, numpy | backtrader, pymysql |
| 脚本参数 | 股票代码 + 周期 + 条数 | 指标名 + 股票代码 | 股票代码 + 日期范围 + 策略名 |
| 超时设置 | 60s | 30s | 120s |
| 平台限制 | 仅 Windows（xtquant） | Windows + Linux | Windows + Linux |

3个量化Skill的对比

# 常见问题：技能看不到怎么办？

Thinking：与 AI 对话时，AI 看不到 miniqmt-kline、talib、backtrader 技能，只显示 Feishu、Gemini 等其他技能，怎么办？

技能加载优先级：extra < bundled < managed < agents-skills < workspace。

如果通过全局安装或非项目目录运行，可能无法从 bundled 目录加载到这些技能。

=> 解决方案

1. 将技能复制到工作区：C:\Users\<用户名>\.openclaw\workspace\skills\

2. 在 openclaw.json 中显式指定 workspace：agents.defaults.workspace 和 quant-analyst.workspace 设为 ~/.openclaw/workspace

3. 确认使用 quant-analyst 智能体：在 Dashboard 中切换到「量化分析师」智能体

4. 重启 Gateway：修改配置后需重启

# OpenClaw技能使用（知识类）

backtrader 支持哪些策略？怎么用？

Backtrader 支持以下策略：

1. **双均线 (double_ma)**
   - 逻辑：10日均线与30日均线金叉买入，死叉卖出
   - 适合趋势行情，简单直观
2. **MACD (macd)**
   - 逻辑：MACD线上穿信号线买入，线下穿卖出
   - 捕捉趋势转折点，过滤震荡行情
3. **RSI (rsi)**
   - 逻辑：RSI低于30时买入，高于70时卖出
   - 适合震荡市，避免追高杀跌
4. **布林带 (bbands)**
   - 逻辑：价格触及下轨买入，触及上轨卖出
   - 利用波动率捕捉超卖/超买机会
5. **动量 (momentum)**
   - 逻辑：近期涨幅排名靠前的股票买入
   - 跟踪强势股，捕捉加速上涨行情

**使用方法：**

```
python3 {baseDir}/scripts/run_backtest.py <股票代码> [开始日期] [结束日期] [策略]
```

**示例：**

- 对贵州茅台跑双均线策略回测：
  `python3 run_backtest.py 600519.SH double_ma`
- 用 MACD 策略回测 2024-01-01 至 2025-12-31：
  `python3 run_backtest.py 600519.SH 2024-01-01 2025-12-31 macd`

---

**前提条件：**

- MySQL 数据库中需有 `trade_stock_daily` 表（可用 miniqmt-kline 采集）
- 配置 `.env` 文件（含数据库连接信息和回测参数）
- 安装依赖包：`backtrader`, `pandas`, `pymysql`, `python-dotenv`

需要我帮你执行某个具体回测吗？

# OpenClaw技能使用（知识类）

talib 能计算哪些技术指标？RSI 超买超卖怎么判断？

TALIB 支持 **158 种技术指标**，涵盖均线、动量、波动率、形态识别等类别。常见指标包括：

| 指标 | 说明 | 应用场景 |
| --- | --- | --- |
| **SMA** | 简单移动平均 | 趋势判断（如10日/30日均线） |
| **EMA** | 指数移动平均 | 更敏感的趋势跟踪（如12日/26日EMA） |
| **MACD** | 指数平滑异同移动平均 | 金叉买入/死叉卖出信号 |
| **RSI** | 相对强弱指标 | 超买超卖判断（核心！） |
| **ATR** | 真实波幅均值 | 止损设置（止损=入场价-2×ATR） |
| **BBANDS** | 布林带 | 下轨买入/上轨卖出，波动率监控 |

---

### 🔍 RSI 超买超卖怎么判断？

- **RSI > 70**：超买区 → 可能回调，考虑卖出
- **RSI < 30**：超卖区 → 可能反弹，考虑买入
- **RSI 在 30-70 之间**：震荡行情，需结合其他指标

> 💡 *例如：若某股票 RSI 连续3天高于70，但价格仍在上涨，可能进入"强势超买"阶段，此时可设止盈；若 RSI 低于30且K线出现锤子线，则是潜在底部信号。*

需要我帮你计算某个股票的 RSI 或其他指标吗？

# OpenClaw技能使用（知识类）

miniqmt 获取 K 线需要什么前置条件?

使用 MiniQMT 获取 K 线数据需要以下前置条件：

### ✅ 前置条件清单

1. **已安装并登录 MiniQMT 客户端**
   - 下载地址：迅投 QMT 官网（需注册账号）
   - 登录后保持客户端运行状态（不能关闭！）
2. **xtquant 包已配置到 Python 环境**
   - MiniQMT 安装目录下包含 `xtquant` 文件夹（通常是 `C:\Program Files\MiniQMT\xtquant`）
   - 需将该目录添加到 Python 路径中，或复制到当前项目环境（如用 `pip install -e <path>` 安装）
3. **Python 环境支持**
   - 安装依赖包：`xtquant`, `pandas`
   - 推荐使用 MiniQMT 自带的 Python 环境（路径类似 `C:\Program Files\MiniQMT\python.exe`）

### 📌 使用示例

```
python3 get_kline.py 600519.SH 1d 100
```

- 获取贵州茅台（600519.SH）最近 100 天的日线数据
- 若不指定参数，默认返回 100 条日线数据

> ⚠️ 注意：如果 MiniQMT 未登录或 xtquant 未正确配置，会报错"无法连接行情服务器"或"找不到模块 xtquant"。

需要我帮你检查是否满足这些条件吗？或者直接获取某只股票的K线数据?

# OpenClaw技能使用（知识类）

我想做量化回测，从数据采集到指标计算到回测，你能帮我规划下流程吗？

当然可以！以下是**从数据采集到指标计算再到回测的完整量化流程规划**，专为你量身定制：

### 🧠 一、明确目标（先想清楚你要做什么）

- 想回测哪种策略？双均线？MACD？RSI？动量？
- 目标股票：单只个股（如贵州茅台）还是多股组合？
- 回测周期：过去3年？5年？还是特定行情？

> ✅ 建议：先从一只股票+一个策略开始，跑通流程再扩展。

### 📥 二、数据采集（用 MiniQMT 获取 K 线）

✅ **步骤：**

1. 安装并登录 MiniQMT 客户端（保持运行）
2. 使用脚本获取日线/分钟线数据：

```
python3 get_kline.py <股票代码> [period] [count]
```

示例：
`python3 get_kline.py 600519.SH 1d 1000` → 获取贵州茅台最近100天日线

3. 将数据存入 MySQL 表 `trade_stock_daily`（可用 Python 脚本自动入库）

> 💡 提示：可批量采集多个股票，写个循环脚本效率更高！

### 📊 三、技术指标计算（用 TA-Lib 分析数据）

✅ **步骤：**

1. 读取已入库的 K 线数据（CSV 或数据库）
2. 使用 TA-Lib 计算关键指标：
   - RSI（超买超卖信号）
   - MACD（金叉死叉）
   - 布林带（波动率监控）
   - SMA/EMA（趋势判断）

   示例命令：

```
python3 calc_indicators.py rsi --stock 600519.SH --period 14
```

3. 输出结果可用于后续策略逻辑判断（如"RSI<30且价格低于布林带下轨"时买入）

> 🔍 小技巧：指标计算后保存为新表（如 `trade_stock_indicators`），方便复用！

### 🧪 四、策略回测（用 Backtrader 执行）

✅ **步骤：**

1. 确保 MySQL 中有 `trade_stock_daily` 表（含你采集的数据）
2. 配置 `.env` 文件（数据库连接 + 回测参数）：

```
WUCAI_SQL_HOST=localhost
```

# OpenClaw技能使用

2. 配置 `.env` 文件（数据库连接 + 回测参数）：

```
WUCAI_SQL_HOST=localhost
WUCAI_SQL_USERNAME=root
WUCAI_SQL_PASSWORD=xxx
WUCAI_SQL_DB=wucai_trade
BACKTEST_INITIAL_CASH=1000000
BACKTEST_COMMISSION=0.0002
BACKTEST_POSITION_PCT=95
```

3. 运行回测脚本：

```
python3 run_backtest.py 600519.SH 2024-01-01 2025-12-31 rsi
```

   - 参数说明：股票代码 + 时间范围 + 策略名（如 `rsi`, `macd`, `double_ma`）

4. 查看输出结果：
   - 总收益率、最大回撤、胜率、交易次数
   - 详细交易记录（买入/卖出时间、价格、仓位）

> 🎯 重点：根据回测结果优化策略（比如调整 RSI 参数、加入止损规则等）

---

### 🔄 五、迭代优化（持续改进）

| 阶段 | 动作 |
|---|---|
| 数据层 | 采集更多股票、更长周期数据 |
| 指标层 | 尝试组合指标（如 RSI+MACD）或自定义因子 |
| 策略层 | 加入风控（止损/止盈）、仓位管理（固定比例/动态调整） |
| 回测层 | 多维度验证（不同时间段、不同市场环境） |

---

### 🧰 工具链总结（你已有全部组件！）

| 功能 | 工具 | 作用 |
|---|---|---|
| 数据采集 | MiniQMT | 获取真实历史K线 |
| 指标计算 | TA-Lib | 快速生成技术信号 |
| 回测引擎 | Backtrader | 自动执行策略并评估绩效 |

---

✅ **下一步建议：**
你可以现在就试试这个流程：

1. 用 `get_kline.py` 获取一只股票的日线数据
2. 用 `calc_indicators.py` 计算 RSI 和 MACD
3. 用 `run_backtest.py` 跑 RSI 策略回测

需要我帮你一步步执行某个环节吗？比如直接调用脚本？

# OpenClaw技能使用（执行类）

获取贵州茅台 600519.SH 最近 3 天日线

Here is the recent 3-day daily K-line data for Guizhou Maotai (600519.SH). Each entry includes the date, opening price, highest price, lowest price, closing price, trading volume, and turnover amount. Let me know if you need any specific analysis or further processing of this data.


```
{
  "stock_code": "600519.SH",
  "period": "1d",
  "data_count": 3,
  "data": [
    {
      "date": "2026-03-04",
      "open": 1415.0,
      "high": 1423.0,
      "low": 1392.09,
      "close": 1401.18,
      "volume": 48014,
      "amount": 6743267103.0
    },
    {
      "date": "2026-03-05",
      "open": 1406.0,
      "high": 1412.96,
      "low": 1395.0,
      "close": 1399.04,
      "volume": 30505,
      "amount": 4276226062.0
    },
    {
      "date": "2026-03-06",
      "open": 1395.0,
      "high": 1407.5,
      "low": 1388.0,
      "close": 1402.0,
      "volume": 29154,
      "amount": 4072328833.0000005
    }
  ]
}
```

# OpenClaw技能使用（执行类）

计算 600519.SH 的 MACD 指标

计算完成！以下是贵州茅台 (600519.SH) 最近的 MACD 指标值：

- **MACD**: -2.9282
- **Signal**: 9.8121
- **Histogram**: -12.7403

这些数据可以用于分析趋势和交易信号。如果需要进一步的解释或可视化，请告诉我！

# OpenClaw技能使用（执行类）

双均线策略回测 600519.SH（2025-01-01 至 2025-12-31）

双均线策略回测结果（600519.SH，2025-01-01 至 2025-12-31）：

- **交易日总数**：243天
- **初始资金**：¥1,000,000
- **最终价值**：¥910,551.59
- **总收益率**：-8.94%
- **年化收益率**：-9.26%
- **最大回撤**：10.36%
- **夏普比率**：0
- **总交易次数**：4次
- **胜率**：25%

看起来这个策略在这段时间内表现不佳。我们可以尝试调整参数或测试其他策略。需要进一步优化吗？

# OpenClaw的调用逻辑

```
{
  "gateway": {
    "mode": "local",
    "port": 18789,
    "auth": { "token": "${OPENCLAW_GATEWAY_TOKEN}" }
  },
  "models": {
    "mode": "merge",
    "providers": {
      "dashscope": {
        "baseUrl": "https://dashscope.aliyuncs.com/compatible-mode/v1",
        "apiKey": "${OPENAI_API_KEY}",
        "api": "openai-completions",
        "models": [
          { "id": "qwen-turbo", "name": "Qwen Turbo" },
          { "id": "qwen-turbo-latest", "name": "Qwen Turbo Latest" },
          { "id": "qwen-max-latest", "name": "Qwen Max Latest" }
        ]
      }
    }
  },
  "agents": {
    "defaults": {
      "model": { "primary": "dashscope/qwen-turbo" },
      "workspace": "~/.openclaw/workspace"
    },
    "list": [
      {
        "id": "quant-analyst",
        "model": "dashscope/qwen-turbo-latest",
        "workspace": "~/.openclaw/workspace",
        "skills": ["miniqmt-kline", "talib", "backtrader"]
      }
    ]
  }
}
```

配置文件：openclaw.json

# OpenClaw的调用逻辑

Thinking：针对刚才的 “获取贵州茅台 600519.SH 最近 3 天日线” 为例，完整调用链路是怎样的?

Step1，用户发送消息

用户 --> Gateway (ws://127.0.0.1:18789)

消息："获取贵州茅台 600519.SH 最近 3 天日线"

Step2，Gateway 将消息转发给 Agent

Gateway --> Agent (quant-analyst)

构建 System Prompt：

- 基础人设（IDENTITY.md, SOUL.md）
- 工具列表（exec, read, write...）
- 技能摘要（3 个 SKILL.md 的描述）

# OpenClaw的调用逻辑

Step3，LLM 分析并匹配技能

LLM 看到 miniqmt-kline 的 description：

"获取 A 股 K 线...命令 python

skills/miniqmt-kline/scripts/get_kline.py 600519.SH 1d 50"

=> LLM 决定调用 exec 工具，命令：

python skills/miniqmt-kline/scripts/get_kline.py 600519.SH 1d 3

Step4，exec 工具执行 Python 脚本

exec 在工作区目录下执行命令

cwd: C:\Users\xxx\.openclaw\workspace

cmd: python skills/miniqmt-kline/scripts/get_kline.py 600519.SH 1d 3

# OpenClaw的调用逻辑

Step5，脚本返回 JSON，LLM 整理后回复用户

get_kline.py 输出:

{"stock_code": "600519.SH",

"data": [{"date": "2026-03-04", ...}]}

LLM 将 JSON 整理成自然语言回复给用户

# 技能匹配机制

Thinking：OpenClaw 的技能匹配机制是怎样的?

- 匹配并不是关键词匹配，而是依赖 LLM 的理解能力：
- Gateway 启动时，读取 Agent 配置的 skills 列表
- 将每个 SKILL.md 的 description 字段注入到 System Prompt 中
- 用户发消息时，LLM 根据消息内容和 Skill 描述，自动决定调用哪个技能
- LLM 通过 exec 工具发起 shell 命令，执行对应的 Python 脚本

description 字段非常关键，它是 LLM 判断是否使用该技能的首要依据。

# 常见问题梳理

Q1：{baseDir} 占位符不生效

问题：早期 SKILL.md 中使用 {baseDir}/scripts/get_kline.py，期望运行时替换为实际路径，但 OpenClaw 并不支持这个占位符。

解决：改为工作区相对路径 skills/miniqmt-kline/scripts/get_kline.py，因为 exec 的工作目录就是 ~/.openclaw/workspace。

# 常见问题梳理

## Q2：AI 反复执行 pip install

问题：

AI 看到 requires.packages 中声明了 xtquant，就自作主张去 pip install xtquant

解决：

- 在 description 和正文中显式写上：禁止 pip install、禁止检查环境
- 添加 执行流程（必须遵守）章节约束 AI 行为。

# 常见问题梳理

## Q3：python3 在 Windows 上不存在

问题：SKILL.md 中 requires.bins 写了 python3，Windows 上默认只有 python 命令。

解决：将 requires.bins 改为 ["python"]，命令中也改为 python。

## Q4：.env 文件不被 Gateway 加载

问题：项目根目录的 .env 文件中配置了 OPENAI_API_KEY，但 Gateway 启动时并没有加载它，导致 API Key 401 错误。

解决：将 API Key 直接写入 openclaw.json 的 apiKey 字段（openclaw.json 不提交到 git，不会泄露）。

# 常见问题梳理

## Q5：工作区与项目目录不同步

问题：修改了项目中的 skills/xxx/SKILL.md，但 Agent 实际读取的是 ~/.openclaw/workspace/skills/xxx/SKILL.md，两者不同步。

解决：每次修改 Skill 后，执行同步命令：

```
xcopy /E /Y "skills\miniqmt-kline\*" "%USERPROFILE%\.openclaw\workspace\skills\miniqmt-kline\"
xcopy /E /Y "skills\talib\*" "%USERPROFILE%\.openclaw\workspace\skills\talib\"
xcopy /E /Y "skills\backtrader\*" "%USERPROFILE%\.openclaw\workspace\skills\backtrader\"
```

# 常见问题梳理

Q6： Backtrader 脚本找不到 .env

问题：run_backtest.py 默认从当前工作目录加载 .env，但 Agent 通过 exec 执行时工作目录是 ~/.openclaw/workspace，该目录下没有数据库凭据。

解决：修改 load_env() 函数，增加从脚本所在技能目录查找 .env 的逻辑，并在 skills/backtrader/.env 中配置数据库凭据。

# Summary

获取贵州茅台 600519.SH 最近 3 天日线

AI 实际执行的命令： python skills/miniqmt-kline/scripts/get_kline.py 600519.SH 1d 3

返回结果：


```
{
  "stock_code": "600519.SH",
  "period": "1d",
  "data_count": 3,
  "data": [
    {"date": "2026-03-04", "open": 1410.0, "high": 1418.0, "low": 1398.0, "close": 1405.0, "volume": 28456, "amount": 3985632000.0},
    {"date": "2026-03-05", "open": 1405.0, "high": 1412.0, "low": 1395.0, "close": 1398.0, "volume": 26789, "amount": 3756123000.0},
    {"date": "2026-03-06", "open": 1395.0, "high": 1407.5, "low": 1388.0, "close": 1402.0, "volume": 29154, "amount": 4072328833.0}
  ]
}
```

# Summary

计算 600519.SH 的 MACD 指标

AI 实际执行的命令： python skills/talib/scripts/calc_indicators.py macd --stock 600519.SH

返回结果：


```
{
  "macd": [-7.91, -9.60, ..., -2.93],
  "signal": [-6.30, -6.96, ..., 9.81],
  "histogram": [-1.61, -2.64, ..., -12.74],
  "length": 200,
  "last_values": {"macd": -2.9282, "signal": 9.8121, "histogram": -12.7403}
}
```

# Summary

双均线策略回测 600519.SH（2025-01-01 至 2025-12-31）

AI 实际执行的命令： python skills/backtrader/scripts/run_backtest.py 600519.SH 2025-01-01 2025-12-31 double_ma

返回结果：


```
{
  "stock_code": "600519.SH",
  "strategy": "double_ma",
  "start_date": "2025-01-01",
  "end_date": "2025-12-31",
  "trading_days": 243,
  "initial_cash": 1000000,
  "final_value": 910551.59,
  "total_return": -0.0894,
  "annual_return": -0.0926,
  "max_drawdown": 0.1036,
  "sharpe_ratio": 0,
  "total_trades": 4,
  "win_rate": 0.25
}
```


AI 会将这些 JSON 数据整理成自然语言回复，比如："双均线策略回测已完成，初始资金 100 万，最终价值 91.06 万，总收益率 -8.94%，最大回撤 10.36%..."

# 如何新增一个 Skill

Thinking：如何新增一个 Skill

Step1，创建技能目录和脚本

```
skills/
  stock-screener/
    SKILL.md
    scripts/
      screener.py
```

# 如何新增一个 Skill

Step2，编写 SKILL.md

```
---
name: stock-screener
description: "A 股选股筛选。直接执行 exec，命令
python skills/stock-screener/scripts/screener.py ..."
metadata:
  openclaw:
    requires:
      bins: ["python"]
    permissions: ["exec"]
    timeout: 60
```

```
---
# 选股技能

## 执行流程（必须遵守）
1. 收到选股请求后，直接执行 exec
2. 命令： python skills/stock-screener/scripts/screener.py <
条件参数>
3. 将脚本输出的 JSON 直接返回给用户

禁止：安装依赖、检查环境。
```

# 如何新增一个 Skill

Step3，注册到 openclaw.json

"skills": ["miniqmt-kline", "talib", "backtrader", "stock-screener"]

| 文件 | 位置 | 用途 |
| --- | --- | --- |
| openclaw.json | 项目根目录 | 配置 AI 模型、Agent、技能列表 |
| skills/xxx/SKILL.md | 每个技能目录 | 定义技能元数据和 AI 操作手册 |
| skills/xxx/scripts/*.py | 每个技能目录 | 实际执行的 Python 脚本 |
| skills/backtrader/.env | backtrader 技能目录 | MySQL 数据库凭据 |
| ~/.openclaw/workspace/ | 用户 home 目录 | Agent 运行时的工作区（需同步） |

关键配置文件清单

# Summary

搭建一个量化交易 AI 同事的核心步骤：

1. 写 Python 脚本：封装量化功能（数据获取、指标计算、回测），统一用 JSON 输出
2. 写 SKILL.md：告诉 AI 这个脚本干什么、怎么调用、参数是什么
3. 注册到 openclaw.json：把技能挂载到指定 Agent
4. 同步到工作区：确保 ~/.openclaw/workspace/skills/ 下有最新文件
5. 启动 Gateway 测试：通过对话验证 AI 能正确调用脚本并返回结果

AI 在这个体系中扮演的是调度员角色，即理解用户的自然语言请求，匹配到合适的技能，拼装出正确的命令行参数，执行脚本，再将结果翻译成用户能理解的语言。

=> 真正的量化逻辑都在 Python 脚本里。

# 打卡：打造你的量化交易同事

基于OpenClaw打造量化交易技能

- 配置OpenClaw运行环境
- 编写 Python 脚本，封装量化功能（数据获取、指标计算、回测）
- 编写 SKILL.md
- 注册到 openclaw.json
- 同步到工作区
- 启动 Gateway 测试

# Thank You

## Using data to solve problems